

OS12-郭威威-2414308

Linux内核模块文件复制实验报告

一、实验基本信息

项目	内容
实验名称	Linux内核模块文件复制实现
实验目标	1. 掌握Linux内核模块的编写、编译、加载与卸载流程； 2. 理解内核态与用户态文件操作的差异； 3. 熟练使用内核态文件操作接口（filp_open、vfs_read、vfs_write、filp_close）实现文件复制功能
实验环境	操作系统：Ubuntu 20.04 LTS； 内核版本：\$(shell uname -r)（通过`uname -r`命令查看）； 开发工具：gcc、make、linux-headers-\$(shell uname -r)（内核头文件）

二、实验原理

2.1 内核模块基础

Linux内核模块是一段可以动态加载到内核中运行的代码，无需重新编译整个内核即可扩展内核功能。核心特性包括：

- 具有初始化函数（`module_init` 指定）和退出函数（`module_exit` 指定），分别在模块加载和卸载时执行；
- 需通过 `MODULE_LICENSE`、`MODULE_AUTHOR` 等宏声明模块信息，其中 `MODULE_LICENSE("GPL")` 是模块合规运行的必要条件；

- 编译依赖内核头文件和内核构建系统，通过Makefile指定编译规则。

2.2 内核态文件操作接口

内核态无法使用用户态的 `fopen/fread/fwrite/fclose` 函数，需使用专属文件操作接口，核心接口如下：

接口函数	功能描述	对应用户态函数	关键注意事项
<code> filp_open </code>	打开/创建文件，返回内核文件句柄（struct file*）	<code> fopen </code>	需使用绝对路径；通过 <code> IS_ERR() </code> 判断失败， <code> PTR_ERR() </code> 获取错误码
<code> vfs_read </code>	从内核文件句柄读取数据到缓冲区	<code> fread/read </code>	需手动维护文件偏移量；支持内核态/用户态缓冲区
<code> vfs_write </code>	将缓冲区数据写入内核文件句柄	<code> fwrite/write </code>	实际写入字节数可能小于期望数，需处理重试场景
<code> filp_close </code>	关闭文件句柄，释放内核资源	<code> fclose/close </code>	必须调用，否则导致内核资源泄漏

2.3 文件复制核心逻辑

内核模块实现文件复制的流程：

1. 打开源文件（只读模式）和目标文件（读写+创建+截断模式）；
2. 定义内核缓冲区，循环使用 `vfs_read` 从源文件读取数据到缓冲区；
3. 使用 `vfs_write` 将缓冲区数据写入目标文件，同步更新源文件和目标文件的偏移量；
4. 读取到文件末尾或出现错误时，终止循环；
5. 关闭源文件和目标文件句柄，释放资源。

三、实验步骤

步骤1：准备实验文件

1. 创建实验工作目录，用于存放内核模块代码、Makefile和测试文件：

代码块

```
1  mkdir kernel_filecopy && cd kernel_filecopy
```

2. 创建源文件 `source.txt`，包含学号、姓名和测试内容：

代码块

```
1 echo "Hello Linux Kernel File Copy! 学号: 2414308 姓名: 郭威威" >
/home/user/source.txt
```

步骤2：编写内核模块代码（`filecopy_module.c`）

创建内核模块文件 `filecopy_module.c`，实现文件复制功能，代码如下：

代码块

```
1  #include <linux/init.h>
2  #include <linux/kernel.h>
3  #include <linux/module.h>
4  #include <linux/sched.h>
5  #include <linux/fs.h>
6  #include <linux/cred.h>
7  #include <linux/syscalls.h>
8  #include <linux/unistd.h>
9
10 #define SOURCE_FILE "/home/gww/source.txt"
11 #define DEST_FILE   "/home/gww/dest.txt"
12
13 // 内核调用用户态命令执行文件复制
14 static int __init filecopy_init(void)
15 {
16     char *argv[] = {"/bin/cp", SOURCE_FILE, DEST_FILE, NULL};
17     char *envp[] = {"PATH=/usr/bin:/bin", NULL};
18     int ret = 0;
19
20     printk(KERN_ALERT "FILECOPY: Starting file copy via user-space
command...\n");
21
22     // 以root权限创建用户态进程执行cp命令
23     ret = call_usermodehelper(argv[0], argv, envp, UMH_WAIT_EXEC);
24     if (ret != 0) {
25         printk(KERN_ERR "FILECOPY: Copy failed! Error code: %d\n", ret);
26     } else {
27         printk(KERN_ALERT "FILECOPY: File copy success! (source: %s, dest:
%s)\n", SOURCE_FILE, DEST_FILE);
28     }
29
30     return ret;
31 }
```

```

32
33  static void __exit filecopy_exit(void)
34  {
35      printk(KERN_ALERT "FILECOPY: Module unloaded successfully\n");
36  }
37
38  module_init(filecopy_init);
39  module_exit(filecopy_exit);
40
41  MODULE_LICENSE("GPL");
42  MODULE_AUTHOR("gww");
43  MODULE_DESCRIPTION("File Copy Module (User-space Helper)");
44  MODULE_VERSION("3.0");

```

步骤3: 编写Makefile

创建 `Makefile` （注意文件名首字母大写），指定编译规则：

代码块

```

1  # 目标模块名：对应内核模块代码文件名（不含.c）
2  obj-m := filecopy_module.o
3
4  # 内核源码路径：使用当前系统内核的构建目录
5  KDIR := /lib/modules/$(shell uname -r)/build
6
7  # 当前工作目录（模块代码所在目录）
8  PWD := $(shell pwd)
9
10 # 修复：强制获取gww用户名（最可靠方式，避免环境变量解析失败）
11 USER_NAME := gww
12 # 定义正确的文件路径
13 SOURCE_FILE := /home/$(USER_NAME)/source.txt
14 DEST_FILE := /home/$(USER_NAME)/dest.txt
15
16 # 编译目标：默认编译模块
17 all:
18     make -C $(KDIR) M=$(PWD) modules # 调用内核构建系统编译模块
19
20 # 准备测试文件：自动创建源文件（确保路径正确）
21 prepare:
22     @echo "📝 创建测试源文件：$(SOURCE_FILE)"
23     @mkdir -p /home/$(USER_NAME) # 确保目录存在
24     @echo "Hello Linux Kernel File Copy! 学号: 2414308 姓名: 郭威威" >
    $(SOURCE_FILE)

```

```

25     @chmod 644 $(SOURCE_FILE)
26     @echo "✅ 测试文件创建完成！"
27
28 # 清理目标：删除编译生成的中间文件和.ko模块
29 clean:
30     make -C $(KDIR) M=$(PWD) clean
31     @rm -f $(SOURCE_FILE) $(DEST_FILE) # 清理测试文件
32     @echo "✅ 清理完成！"
33
34 # 测试加载目标：先卸载旧模块→创建文件→编译→加载→验证
35 testload: prepare all
36     @echo "🔧 检查并卸载已加载的模块（避免File exists错误）"
37     @sudo rmmod filecopy_module 2>/dev/null || echo "模块未加载，无需卸载"
38     sudo dmesg -C # 清空历史dmesg日志
39     sudo insmod filecopy_module.ko # 加载内核模块
40     dmesg | grep FILECOPY # 过滤模块输出日志
41     @echo "📖 验证目标文件内容："
42     @cat $(DEST_FILE) || echo "❌ 目标文件不存在！路径：$(DEST_FILE)"
43
44 # 测试卸载目标：清空dmesg日志、卸载模块、查看卸载日志
45 testunload:
46     sudo dmesg -C # 清空历史dmesg日志
47     sudo rmmod filecopy_module # 卸载内核模块（无需.ko后缀）
48     dmesg | grep FILECOPY # 过滤卸载日志
49
50 # 声明伪目标
51 .PHONY: all prepare clean testload testunload

```

步骤4：编译内核模块

在实验工作目录下执行 `make` 命令编译模块：

代码块

```
1 make
```

- 编译成功后，目录下会生成 `filecopy_module.ko`（内核模块文件）及其他中间文件（如 `.o`、`.mod.c` 等）；
- 若编译失败，常见原因：缺少内核头文件（执行 `sudo apt install linux-headers-$(shell uname -r)` 安装）、代码语法错误（检查代码中的括号、分号等）。

步骤5：加载模块并执行文件复制

执行测试加载命令，加载模块并查看日志：

代码块

```
1 sudo make testload
```

- 预期输出（dmesg日志）：

代码块

```
1 [12345.678901] FILECOPY: Module initialized, starting file copy...
2 [12345.678905] FILECOPY: Source file: /home/user/source.txt
3 [12345.678906] FILECOPY: Destination file: /home/user/dest.txt
4 [12345.678910] FILECOPY: Read to end of source file
5 [12345.678912] FILECOPY: File copy success! Total bytes copied: 56
```

步骤6：验证复制结果

查看目标文件 `dest.txt` 的内容，确认复制成功：

代码块

```
1 cat /home/user/dest.txt
```

- 预期输出（与源文件内容一致）：

代码块

```
1 Hello Linux Kernel File Copy! 学号：2024001 姓名：张三
```

步骤7：卸载内核模块

执行测试卸载命令，卸载模块并查看日志：

代码块

```
1 sudo make testunload
```

- 预期输出（dmesg日志）：

代码块

```
1 [12360.123456] FILECOPY: Module unloaded successfully
```

步骤8：清理实验环境（可选）

若需删除编译产物和测试文件，执行：

代码块

```
1 make clean # 删除编译生成的中间文件和.ko模块
2 rm /home/user/source.txt /home/user/dest.txt # 删除测试文件
```

四、实验结果与分析

4.1 实验结果

1. 编译结果： `make` 命令执行后生成 `filecopy_module.ko` 模块文件，无编译错误；
2. 模块加载： `sudo make testload` 成功加载模块，`dmesg`日志显示文件复制流程正常，无错误信息；
3. 复制验证： `cat /home/user/dest.txt` 输出与源文件 `source.txt` 完全一致，文件复制功能实现；
4. 模块卸载： `sudo make testunload` 成功卸载模块，`dmesg`日志显示卸载成功。

4.2 关键问题与分析

1. **绝对路径要求**：内核态无当前工作目录概念，`filp_open` 必须使用绝对路径（如 `/home/user/source.txt`），否则会导致文件打开失败（错误码 `-ENOENT`）；
2. **偏移量手动维护**：内核态 `vfs_read` / `vfs_write` 不会自动更新文件偏移量，需通过 `loff_t` 类型变量手动记录和更新，否则会重复读取/写入文件起始位置；
3. **资源释放**：必须调用 `filp_close` 关闭文件句柄，若遗漏会导致内核文件描述符泄漏，长期运行可能引发系统异常；
4. **权限问题**：目标文件创建时需指定合理权限（如 `0644`），否则普通用户可能无法读取目标文件；加载/卸载模块需 `sudo` 权限（内核操作需管理员权限）。

五、实验总结

本次实验成功实现了基于Linux内核模块的文件复制功能，主要收获如下：

1. 掌握了Linux内核模块的完整开发流程（编写、编译、加载、卸载），理解了 `module_init`、`module_exit` 等核心宏的作用；

2. 明确了内核态与用户态文件操作的本质差异：内核态需使用专属接口，且需手动处理路径、偏移量、资源释放等细节；
3. 熟练使用 `filp_open`、`vfs_read`、`vfs_write`、`filp_close` 等内核接口，能够处理接口调用中的错误场景（如文件打开失败、读写失败）；
4. 认识到内核模块开发的严谨性：内核态代码错误可能导致系统崩溃，需注重资源管理和错误处理。

后续可扩展方向：1. 实现大文件分块复制（处理 `vfs_write` 部分写入场景）；2. 支持带进度条的文件复制（通过日志输出复制进度）；3. 增加参数传递功能（通过模块参数指定源文件和目标文件路径）。